
Contenido

- 1 Estado actual del proyecto
- 2 Cómo funciona el sistema
- 3 Tecnologías y arquitectura actual
- 4 Roles del sistema
- 5 Rutas y módulos principales
- 6 Organización del código
- 7 Base de datos
- 8 Seguridad y autenticación
- 9 Pendientes y puntos a corregir

1 Estado actual del proyecto

ENCUADRES-FCQI es una aplicación web para administrar el trabajo académico de cada periodo. Incluye materias, PUA, encuadres, asignaciones docente-grupo, alumnos, firmas, evidencias y seguimiento de avances. En el estado revisado hay cinco perfiles de usuario y alrededor de treinta pantallas que ya forman parte del flujo de trabajo.

Se consultó directamente la instancia activa de Supabase.

Situación actual. La aplicación ya cubre buena parte de los flujos principales, pero todavía no conviene tratarla como una versión de producción. Antes hay que cerrar los temas de autorización y acceso a datos, dejar la base versionada y probar los flujos principales de cada rol.

Área	Estado	Comentarios
Funcionalidad	Bastante avanzada	Los flujos principales ya existen para administración, captura, profesores, alumnos y consulta.
Seguridad	Requiere atención inmediata	RLS está desactivado en public, hay consultas desde cliente con anon y las operaciones con service_role necesitan controles de servidor bien definidos.
Base de datos	Esquema conocido, pero no reproducible todavía	La estructura actual se pudo revisar directamente, pero faltan migraciones y todavía no se tiene el inventario completo de grants, índices y enums.
Calidad verificable	Pendiente de validar	No se encontraron pruebas automatizadas ni un flujo de CI documentado.
Mantenibilidad	Mejorable	Hay módulos grandes, uso frecuente de any, logs dispersos y partes de estructuras anteriores que siguen coexistiendo con el flujo actual.
Preparación para producción	Todavía no	Antes de usarlo como versión de producción hay que resolver los puntos críticos y hacer pruebas de aceptación por rol.

Antes de seguir desarrollando

1. Revisar las rutas administrativas del servidor y validar sesión, rol y acceso al recurso antes de ejecutar operaciones con service_role.
2. Definir cómo se va a manejar la identidad entre NextAuth y Supabase. Mientras el navegador no tenga una sesión válida de Supabase Auth, no conviene depender de auth.uid() para autorizar operaciones.
3. Revisar primero grants y políticas y, después, activar RLS de manera controlada. Cada operación debe probarse con los roles que realmente usa la aplicación.
4. Cerrar el acceso general al Storage de evidencias y limitarlo por usuario, rol o encuadre; cuando aplique, usar URLs firmadas.
5. Pasar la estructura de la base a migraciones versionadas para que tablas, constraints, funciones, triggers, políticas y buckets puedan reconstruirse desde el repositorio.
6. Revisar el sistema de notificaciones y dejar un solo flujo. Las funciones antiguas pueden retirarse o aislarse cuando se confirme que ya no son usadas por la aplicación.

2 Como funciona el sistema

ENCUADRES-FCQI organiza el trabajo académico de cada periodo desde la preparación inicial de la información hasta el seguimiento de los avances de cada curso. El sistema está dividido por roles, de modo que cada usuario accede únicamente a las funciones relacionadas con sus actividades.

El flujo comienza con el administrador, quien prepara la información necesaria para trabajar durante el periodo. Desde este módulo se gestionan usuarios, roles, licenciaturas, materias, periodos, ventanas de operación y permisos especiales. También se pueden revisar resultados generales, configurar fechas de trabajo y publicar anuncios dentro del sistema.

Una vez configurado el periodo, el capturista se encarga de registrar la información académica de las materias. Aquí se captura el Programa de Unidad de Aprendizaje (PUA), incluyendo los datos generales del programa, unidades, temas, competencias, prácticas de taller, prácticas de laboratorio y demás información necesaria para la planeación de la materia.

Después se genera el encuadre correspondiente a cada curso. El encuadre se relaciona con el programa, profesor, grupo y periodo, y contiene información como los criterios de evaluación, criterios de acreditación, normas de conducta, bibliografía, descripción de productos y otras reglas que se presentan a los alumnos al inicio del curso.

Cuando el curso ya está configurado, el profesor trabaja desde su panel con los grupos que tiene asignados. Desde ahí puede consultar el encuadre, gestionar alumnos, registrar evidencias de presentación, revisar firmas y dar seguimiento al avance de los temas del programa. El sistema también conserva información relacionada con los temas revisados y el progreso registrado durante el periodo.

Los alumnos tienen acceso a los cursos en los que fueron registrados. Desde su cuenta pueden consultar la información del encuadre, revisar el plan de trabajo, registrar su firma de enterado y participar en el seguimiento de los avances. Esto permite posteriormente comparar la información registrada por el profesor con la percepción o avance reportado por los alumnos.

El administrador y el lector cuentan con módulos orientados a supervisión. Estos permiten consultar los cursos, revisar información consolidada y comparar los registros de avance entre profesores y alumnos. La finalidad de estos módulos es facilitar el seguimiento académico y detectar diferencias en el desarrollo de los temas planeados.

Además de los flujos principales, el sistema incluye funciones auxiliares como configuración de fechas de operación, permisos temporales, anuncios, notificaciones y almacenamiento de evidencias. Parte de estas funciones depende directamente de Supabase, donde se almacenan los datos académicos, usuarios, relaciones entre cursos, notificaciones y archivos asociados al encuadre.

De forma general, el funcionamiento del sistema puede resumirse en las siguientes etapas:

- 1. ****Preparación del periodo:**** se registran usuarios, materias, licenciaturas, roles, fechas y permisos.
- 2. ****Captura académica:**** se registra el PUA, sus unidades, temas y prácticas.
- 3. ****Configuración del encuadre:**** se asigna profesor, grupo, criterios y reglas del curso.
- 4. ****Operación docente:**** el profesor administra alumnos, evidencias, firmas y avances.
- 5. ****Participación del alumno:**** el alumno consulta el encuadre, firma y registra información de seguimiento.
- 6. ****Supervisión:**** administración y lector revisan los avances y comparan los registros del curso.
- 7. ****Apoyo operativo:**** el sistema maneja anuncios, notificaciones, permisos especiales y evidencias almacenadas en Supabase.

Paso	Proceso	Responsable	Resultado
1	Preparación	Administrador	Usuarios, roles, licenciaturas, materias, periodos, ventanas y permisos.
2	Captura académica	Capturista	PUA, unidades, temas, prácticas, encuadre, criterios y asignaciones.
3	Operación del curso	Profesor	Cursos asignados, alumnos, evidencia, firmas y avance temático.
4	Participación	Alumno	Consulta, firma de enterado y registro de percepción/avance.
5	Supervisión	Administrador / lector	Comparación profesor-alumno y seguimiento por curso.

3 Tecnologías y arquitectura actual

Área	Tecnología	Estado / observación
Frontend / servidor	Next.js 15.2.3 - App Router	Pantallas cliente y route handlers en el mismo repositorio.
Lenguaje	TypeScript 5 - strict	Uso extendido de any reduce garantías de tipos.
Interfaz	React 19 / Tailwind CSS 4 / Radix UI	Componentes propios sobre primitivas UI.
Autenticación	NextAuth 4 Credentials / JWT	Identidad web separada de Supabase Auth.
Datos	Supabase JS 2 / PostgreSQL	Cliente anon desde navegador y service_role en API.
Validación	React Hook Form / Zod	Zod instalado; la revisión base no encontró validación consistente en APIs.
Archivos	Supabase Storage	Bucket evidencias-encuadre; policíes actuales permiten CRUD a anon/authenticated por bucket.

Área	Tecnología	Estado / observación
Correo	email_outbox / Resend instalado	No se verificó consumidor activo de outbox ni uso efectivo de Resend.
Impresión	HTML imprimible	Encuadre/PUA imprimibles; no equivale a generación PDF firmada en servidor.

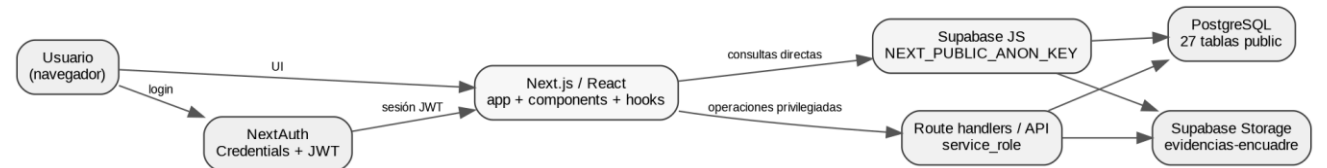


Figura 1. Arquitectura lógica observada y fronteras de identidad/acceso.

Cómo se autentica un usuario

- El inicio de sesión se hace con correo y contraseña desde la aplicación.
- NextAuth busca al usuario en Supabase mediante un cliente con privilegios y compara la contraseña contra el hash almacenado.
- Después del login, el JWT de NextAuth conserva los datos de identidad y rol que usa la navegación y el middleware.
- Una vez dentro, varias pantallas consultan Supabase directamente con la clave pública anon.
- Aquí hay que tener cuidado: auth.uid() pertenece a Supabase Auth y no reconoce por sí solo la sesión que mantiene NextAuth.

Punto importante. La aplicación maneja la sesión de NextAuth por un lado y el contexto de Supabase Auth por otro. Mientras ambos no estén alineados, auth.uid() puede ser NULL o no representar al usuario que inició sesión. Las reglas de autorización deben definirse en un solo lugar y probarse de forma explícita.

4 Roles del sistema

Rol	Propósito	Funciones presentes
Administrador	Configuración global	Usuarios/roles, resultados, materias/licenciaturas, ventanas, anuncios, herramientas y permisos especiales.
Capturista	Captura académica	PUA, unidades, temas, prácticas, encuadres y asignaciones.
Profesor	Operación docente	Cursos asignados, encuadre, alumnos, evidencias, firmas y avances.
Alumno	Consulta y participación	Cursos inscritos, encuadre, firma, plan de clases y avances.
Lector	Supervisión	Dashboard, cursos y comparación agregada en modo de lectura.

5 Rutas y módulos principales

Rol	Ruta	Propósito	Estado
Pública	/	Inicio de sesión	Parcial: recuperación de contraseña sin flujo completo en revisión base.
Admin	/admin	Panel general	Implementado.
Admin	/admin/usuarios	Usuarios, roles e importación	Implementado; requiere autorización robusta del servidor.
Admin	/admin/crear-usuario	Alta alternativa	Incompleto / ruta destino inconsistente en revisión base.
Admin	/admin/materias	Materias, licenciaturas, periodos, archivado	Implementado; módulo grande.

Rol	Ruta	Propósito	Estado
Admin	/admin/resultados	Comparación de avances	Implementado.
Admin	/admin/configurar-fecha	Ventanas de operación	Implementado; DB activa incluye flags para encuadre/firma/avances/PUA.
Admin	/admin/anuncios	Anuncios	UI presente; flujo de triggers activo distribuye a todos los usuarios.
Admin	/admin/herramientas/clonar-periodo	Clonación estructural	Depende de RPC/flujo no verificado entre las funciones públicas extraídas.
Admin	/admin/herramientas/permisos-especiales	Excepciones temporales	Implementado; tabla activa incluye edición de PUA y permisos por recurso.
Capturista	/capturista/materias/[clave]/pua	Datos generales del PUA	Implementado.
Capturista	.../pua/unidades	Unidades y temas	Implementado.
Capturista	.../pua/taller	Prácticas de taller	Implementado.
Capturista	.../pua/laboratorio	Prácticas de laboratorio	Implementado.
Capturista	.../[clave]/encuadre	Encuadre y asignaciones	Implementado.
Profesor	/profesor/cursos/[id]	Encuadre, alumnos, avances, evidencia	Implementado.
Profesor	/profesor/encuadres; /profesor/avances	Flujos anteriores	Posible legado frente a /profesor/cursos.
Alumno	/alumno/cursos/[id]	Encuadre, firma, plan y avances	Implementado.
Lector	/lector/cursos/[encuadre_id]	Listado y detalle	Implementado.
Lector	/lector/reportes	Reportes	Archivo vacío en revisión base.

6 Organización del código

Directorio / archivo	Responsabilidad
app	Rutas, layouts, pantallas y route handlers.
components	Componentes de dominio, impresión, navegación y biblioteca UI.
hooks	Lógica de formularios, acceso a datos, PUA, alumnos, permisos y avances.
lib	Supabase, autenticación, permisos, navegación y utilidades.
public	Recursos gráficos.
schema_actual.txt	Instantánea parcial; ya no debe considerarse fuente canónica del esquema.
supabase/.temp	Metadatos temporales; correctamente debe mantenerse fuera de Git.

Métrica	Valor de revisión base	Interpretación
Archivos TypeScript/TSX	112	Tamaño razonable para el alcance.
Líneas de código	31 510	Lógica concentrada en pantallas y hooks.
Uso de any	96 apariciones	Reduce seguridad de tipos en datos y Excel.

Métrica	Valor de revisión base	Interpretación
console.error	159 apariciones	Falta capa central de observabilidad.
Desactivaciones ESLint	20	Revisar dependencias de hooks caso por caso.
Archivo mayor	admin/materias/page.tsx - 1 173 líneas	Separar servicios, hooks y componentes.

7 Base de datos

La parte de base de datos ya no está documentada solo a partir del código. Se consultó la instancia activa de Supabase y se confirmó el esquema public con 27 tablas y 44 relaciones FOREIGN KEY. El detalle completo de tablas, columnas, funciones, triggers y policies se dejó en el documento específico de base de datos.

Dominio	Tablas	Función
Identidad	usuarios, roles	Usuarios propios, rol y banderas de contraseña.
Catálogo	licenciaturas, materias	Oferta académica, plan de estudios y archivado.
Planeación / PUA	programas, unidades, temas, practicas_taller, practicas_laboratorio, plan_temas	Estructura académica y secuencia temática.
Encuadre	encuadres, criterios_evaluacion, criterios_acreditacion	Acuerdos, criterios y asignación docente/grupo.
Participación	encuadre_alumnos, encuadre_firmas, encuadre_evidencias, encuadre_checkin	Inscripción, firma y evidencias.
Seguimiento	temas_checkin, checkins	Avances y registros de seguimiento.
Operación	configuracion_fechas, configuraciones, permisos_operacion_encuadre	Ventanas, mantenimiento y excepciones temporales.
Comunicación	anuncios, notificaciones, notificaciones_usuarios, email_outbox	Anuncios, distribución y cola de correo.
Auditoría	auditoria	Registro básico de acción, usuario y fecha; su uso debe verificarse.

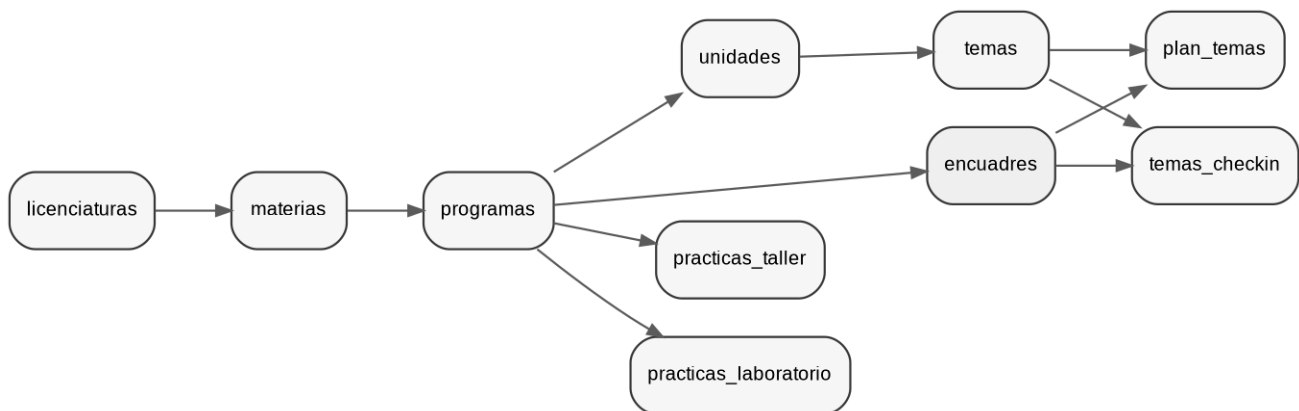


Figura 2. Núcleo relacional académico verificado.

Diferencias encontradas frente a schema_actual.txt

- En la base activa, temas_checkin sí tiene la columna encuadre_id.
- La tabla unidades tiene actualmente las columnas titulo y nombre.
- practicas_taller y practicas_laboratorio mantienen columnas de la estructura anterior junto con las columnas nuevas usadas por el flujo actual.
- materias conserva licenciatura como texto y también licenciatura_id como llave foránea. Esto parece mantenerse por compatibilidad y conviene definir cuál debe ser la fuente principal.

- permisos_operacion_encuadre tiene más campos que la instantánea local: incluye edición de PUA, gestión de alumnos, firma, avances y alcance por programa o encuadre.
- usuarios ya tiene debe_cambiar_password y password_expira_at. Aun así, falta comprobar que el flujo de cambio obligatorio se haga cumplir en toda la aplicación.

8 Seguridad y autenticación

RLS está desactivado en public. Las policies existen, pero mientras RLS siga apagado no están filtrando filas. El acceso real depende de los grants, de la clave utilizada y de las validaciones que haga el servidor.

Policies de notificaciones

Hay policies abiertas, como SELECT true e INSERT WITH CHECK true, junto con otras que intentan limitar por auth.uid(). Si RLS se activa tal como están, las reglas permisivas pueden terminar dejando pasar operaciones que se pretendían restringir.

Storage de evidencias

En storage.objects hay policies para SELECT, INSERT, UPDATE y DELETE sobre el bucket evidencias-encuadre para anon y authenticated. En las reglas revisadas no se valida propietario, profesor, alumno, encuadre ni rol académico.

Funciones con SECURITY DEFINER

- eliminar_programa_periodo(uid) borra un programa y varias dependencias. Por ser SECURITY DEFINER, conviene limitar quién puede ejecutarla y validar el rol desde el servidor.
- mark_notification_read(uid) también es SECURITY DEFINER, usa auth.uid() y apunta a user_notifications. Ese flujo debe revisarse antes de considerarlo parte estable del sistema.

9 Pendientes y puntos a corregir

Área	Prioridad	Acción
Autorización / RLS	P0	RLS apagado; revisar grants, habilitar RLS con policies coherentes y pruebas por rol.
Storage evidencias	P0	Retirar CRUD anónimo general; usar ownership, roles y/o signed URLs.
service_role en APIs	P0	Validar sesión, rol y recurso en cada operación privilegiada.
Notificaciones	P1	Consolidar dos arquitecturas; triggers activos ignoran destinatarios/metodo y distribuyen a todos.
Identidad	P1	Alinear NextAuth con Supabase Auth o eliminar dependencia de auth.uid() en flujos no autenticados con Supabase.
Migraciones	P1	Crear esquema versionado reproducible y eliminar dependencia de snapshot manual.
Contraseña temporal	P1	Completar pantalla/endpoint/reglas y bloqueo de navegación; DB ya contiene banderas.
Correo	P1	Confirmar quién consume email_outbox, idempotencia, reintentos, plantillas y manejo de errores.
Reportes lector	P2	Definir reportes, filtros, exportación y permisos.
Auditoría	P2	Confirmar escrituras; ampliar recurso, antes/después, request ID y actor.
Pruebas	P2	Unitarias, integración y E2E por rol.
Observabilidad	P2	Centralizar errores, métricas y alertas.
Accesibilidad	P2	Auditoría de teclado, foco, etiquetas, contraste y lector de pantalla.

Notas sobre el sistema de notificaciones

- El flujo activo de anuncios crea una notificación y un segundo trigger la asigna a todos los usuarios.

-
- En esa cadena de triggers no se usan destinatarios ni metodo de la tabla anuncios, así que la segmentación guardada en esos campos no se aplica en el flujo activo que se pudo comprobar.
 - La función `fn_fanout_anuncio(anuncio_id)` sí toma en cuenta destinatarios, metodo y email_outbox, pero no está conectada a los triggers activos. Falta revisar si la aplicación la llama directamente como RPC.
 - `fn_fanout_anuncio(anuncio_id)` y `mark_notification_read()` usan `public.user_notifications`. Ese objeto no apareció entre las tablas public revisadas, por lo que hay que confirmar si es una vista, otro tipo de objeto o código que quedó de una implementación anterior.
 - `mark_all_notifications_read()` actualiza `fecha_lectura`, pero no cambia la columna leída. Conviene decidir cuál de las dos se va a usar como estado real de lectura.